EconoCausal

Dynamic Pricing via Double Machine Learning

Project report: Weeks 1 to 4

Project	Project 2: EconoCausal (Infotact Solutions, Advanced Data Science & ML Engineering, Vol. II)
Goal	Estimate how much a discount changes each customer's chance of buying, then spend a fixed budget on the customers it actually moves.
Stack	Python, DoWhy, EconML, scikit-learn, SciPy (Week 3), React, Plotly
Covered	Week 1 (causal graphing, propensity balancing), Week 2 (Double ML, uplift curves, refutation tests), Week 3 (budget optimizer, allocation matrix), Week 4 (REST API, drift detection, dashboard polish)

Why causal and not predictive

A standard model that predicts who is likely to buy will pick customers who would have bought anyway, so the discount is wasted on them. EconoCausal instead estimates the Individual Treatment Effect (ITE): the change in one customer's purchase probability caused by the discount. The budget then goes to the "Persuadables", the customers whose behaviour the discount actually changes.

Status

Week	Backend	Frontend	Status
1	Causal DAG, synthetic data, propensity balancing	Upload and budget views	Done
2	Double ML training (ITE), uplift evaluation, refutation tests	Uplift results tab (Plotly)	Done
3	Revenue-aware SciPy budget optimizer	Allocation matrix	Done
4	REST API, data drift detector, Dockerfile	Monitoring tab, live budget, summaries	Done

Part 1: Week 1

Causal graphing and dashboard scaffolding

Deliverables

- Mock retail dataset with confounders, treatment and outcome (`backend/data/generate_data.py`)
- Causal DAG defined in DoWhy (`backend/causal/build_dag.py`)
- Effect identification by backdoor adjustment (`docs/identified_estimand.txt`)
- Propensity score balancing with overlap and balance plots (`backend/causal/propensity.py`)
- React app with campaign upload and budget views (`frontend/`)

Data and causal setup

The dataset has 5,000 synthetic customers. Because it is simulated, the true effect of the discount on every customer is known, which lets us check later whether the model recovers it.

Role	Variables
Treatment	discount_given
Outcome	purchased
Confounders	past_spend, tenure_months, visits_per_month, price_sensitivity, region

Marketing historically discounted loyal, high-spending customers, so treated and untreated customers differ before any discount is applied. This is confounding: a simple comparison mixes the effect of the discount with the effect of who was targeted. The DAG records this, and adjusting for the confounders removes the bias.

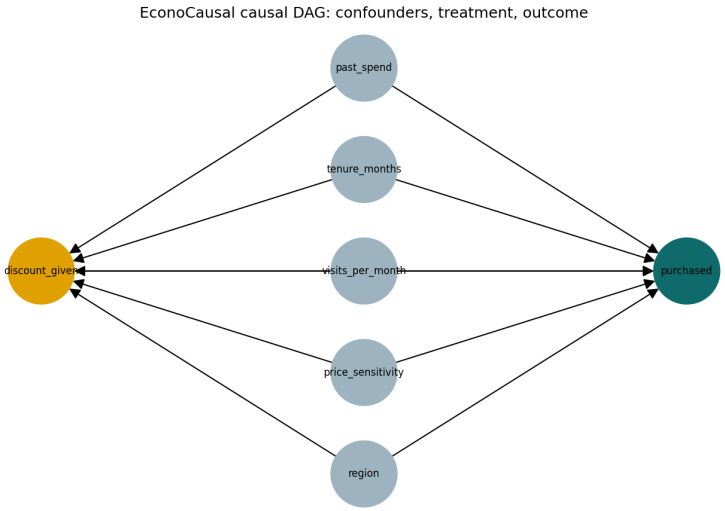


Figure 1. Causal DAG. Every confounder affects both the discount decision and the purchase.

Propensity score balancing

A logistic regression estimates each customer's probability of receiving the discount. Weighting customers by the inverse of that probability makes the two groups comparable, as if the discount had been assigned at random. A covariate is considered balanced when its standardised mean difference (SMD) is below 0.1.

Covariate	SMD before weighting	SMD after weighting
past_spend	+0.748	+0.015
tenure_months	+0.399	-0.016
visits_per_month	+0.289	-0.012
price_sensitivity	-0.004	+0.006
region	+0.039	+0.015

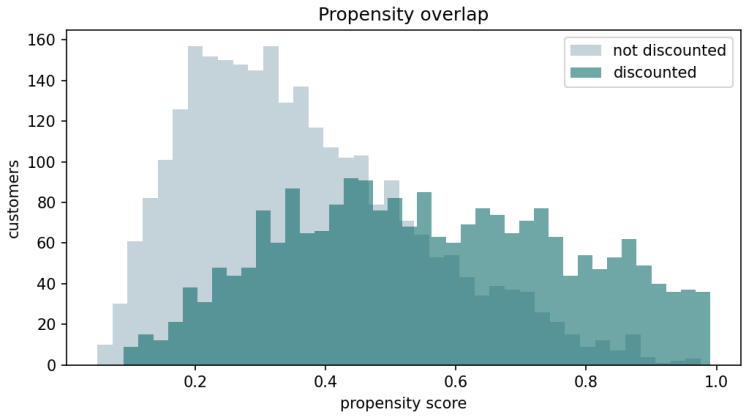


Figure 2. Propensity overlap. Both groups cover the same range, so comparison is valid.

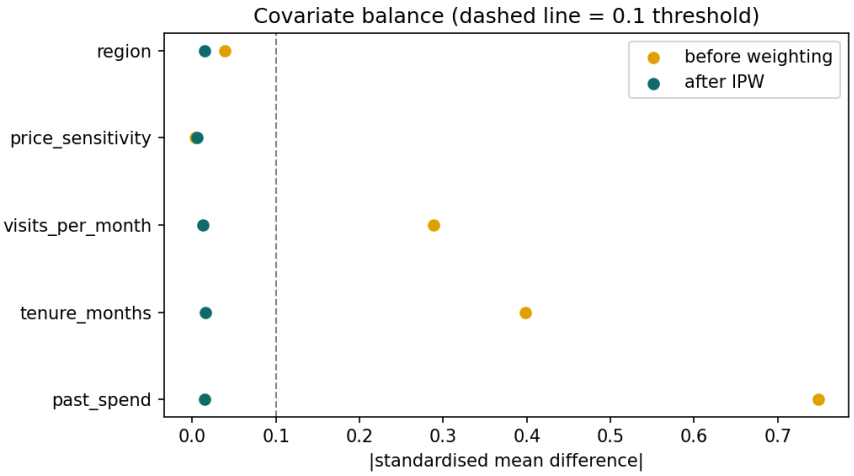


Figure 3. Covariate balance before and after weighting. All covariates fall under the 0.1 line after weighting.

Week 1 results

Estimate of the average discount effect	Value
Naive difference in purchase rate	+0.280
Linear backdoor adjustment	+0.161
Propensity (IPW) adjusted	+0.154
True effect (known from simulation)	+0.125

The naive number is more than double the truth because of the biased targeting. Adjustment brings it much closer. The remaining gap exists because the true effect varies from customer to customer and the relationships are not linear, which is the reason for Double Machine Learning in Week 2.

Dashboard (Week 1)

The React app has two views. The upload view accepts a campaign CSV, checks the required columns (customer_id, discount_given, purchased), and shows a preview and summary statistics. The budget view sets the total discount budget, the discount options and the per-customer cap.

Part 2: Week 2

Double Machine Learning and uplift curves

Deliverables

- Double ML with EconML: LinearDML and CausalForestDML (`backend/causal/train_dml.py`)
- Per-customer effects for 1,500 held-out customers (`backend/data/ite_scores.csv`)
- Qini curves and decile analysis (`backend/causal/uplift.py`)
- Predictive baseline to compare against ordinary machine learning
- DoWhy refutation tests for the Mid Review Causal Audit (`backend/causal/refute.py`)
- React Uplift results tab with Plotly charts (`frontend/src/components/UpliftView.jsx`)

Method

Double ML fits one model that predicts the outcome from the confounders and one that predicts the treatment from the same confounders, using 5-fold cross-fitting so no customer is scored by a model that saw them. It removes what the confounders explain from both, then estimates the effect of the discount from what remains. The causal forest lets that effect vary non-linearly across customers. Random Forest base learners are used. The data is split 70/30, stratified by treatment, and every result below is on the held-out 30%.

Results

Measure	Value
True average effect (simulation)	+0.127
LinearDML average effect	+0.146 (95% CI +0.112 to +0.180)
CausalForestDML average effect	+0.150
Correlation with true per-customer effect	linear 0.86, forest 0.80, predictive baseline -0.28
RMSE vs true per-customer effect	linear 0.056, forest 0.057

The confidence interval for the average effect contains the true value. The predictive baseline is negatively correlated with the true effect: customers who are likely to buy are the ones the discount helps least.

Extra purchases from discounting the best-ranked 30% of customers (ground truth):

Targeting method	Extra purchases
Oracle (ranked by the true effect)	101.8
Linear DML	96.9
Causal forest (DML)	93.6
Random rollout	57.0
Predictive baseline (likely buyers)	44.0

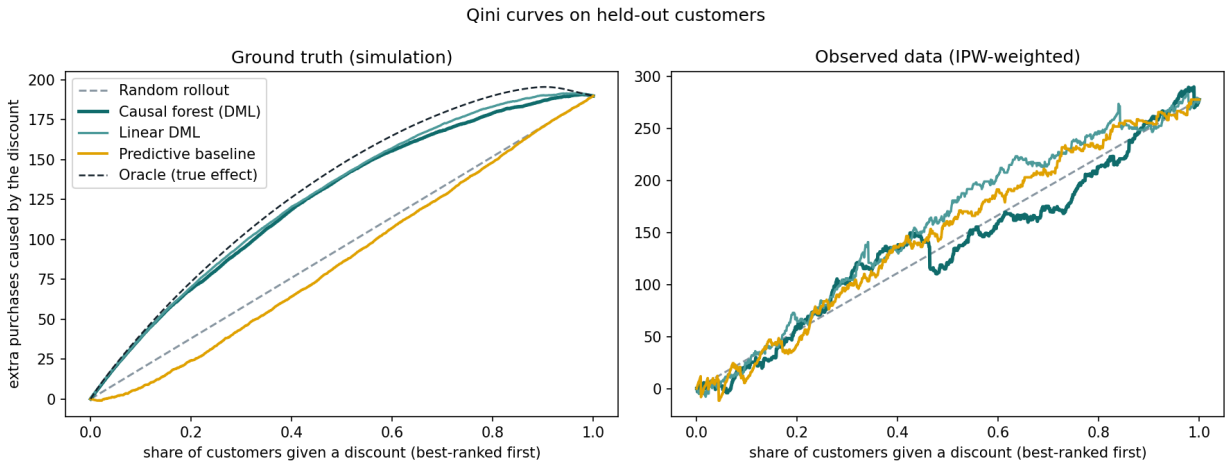


Figure 4. Qini curves. Left: ground truth from the simulation. Right: observed data only, weighted by propensity (noisier). A line above the dashed random line beats guessing.

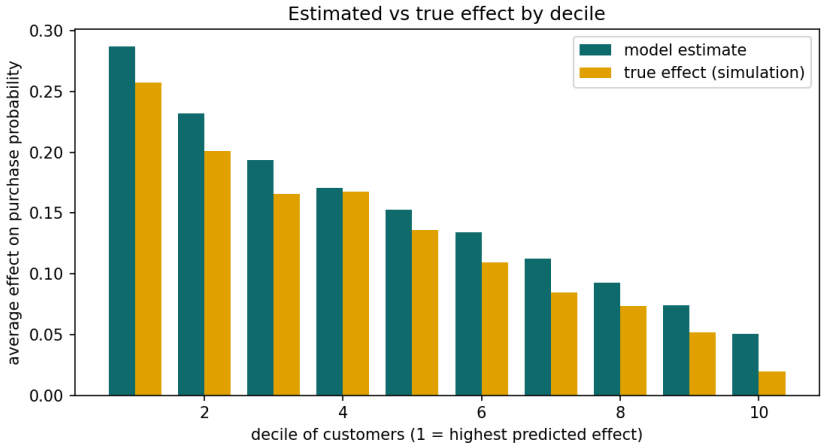


Figure 5. Customers sorted by predicted effect and split into ten groups. The model estimate tracks the true effect and falls from the first group to the last.

Causal audit: refutation tests

Refutation tests try to break the estimate. If it survives, that is evidence the estimate reflects a real causal effect.

Test	Original effect	New effect	Reading
Placebo treatment	+0.161	-0.001	Effect disappears with a fake treatment
Random common cause	+0.161	+0.161	Unchanged by an irrelevant confounder
Data subset (80%)	+0.161	+0.160	Stable across random subsets

Limitations

- The linear model matches or slightly beats the forest here because the simulated effect is close to linear. The forest should do better on data with more complex effects.
- The observed-data Qini curve is noisy at 1,500 customers. The forest beats random in the top 30% there but not clearly over the full curve, so the ground-truth curve is the cleaner proof.
- The data is simulated. Real campaign data will be harder to interpret.

Part 3: Week 3

Prescriptive optimization and the allocation matrix

Deliverables

- Revenue-aware optimizer with a hard budget limit (`backend/optimize/engine.py`)
- Script that prescribes a discount for every customer and compares strategies (`backend/optimize/prescribe.py`)
- Dashboard Allocation tab: allocation matrix with filters, sorting and a budget chart

From effects to action

Week 2 gave an effect for each customer. Week 3 decides who gets which discount ($10, $20 or none) so net revenue is highest without exceeding the budget. For a discount of d dollars: extra revenue = effect x share(d) x (order value - d); wasted cost = baseline purchase chance x d, because customers who would have bought anyway still receive the discount; net gain = extra revenue - wasted cost.

The wasted cost is why the causal effect matters. A Sure thing has a high baseline chance, so the discount mostly gives money away and the net gain is negative. The optimizer skips those customers, so it may spend less than the whole budget. It is solved exactly as a mixed-integer linear program: each customer gets at most one tier, spend stays within budget, and total net gain is maximised.

Assumptions

- A $10 discount achieves 70% of the effect of a $20 discount (diminishing returns). This is assumed, not measured.
- Baseline purchase chance comes from a logistic model fitted on customers who did not get a discount.
- Revenue per purchase equals the customer's average monthly spend.

Results: 1,500 held-out customers, $5,000 budget

Strategy	Customers discounted	Estimated net revenue	True net revenue	True extra purchases
Optimizer	481	$4,643	$2,365	56.4
Blanket $20 rollout	250	$288	-$232	33.3
Likely-buyers targeting	250	$26	-$2,443	10.7

A flat discount, or discounting the customers most likely to buy, loses money. The optimizer earns a clear profit. Its own estimate ($4,643) is higher than the true value ($2,365), because it picks the customers the model rates highest, which includes some the model overrated. The true figure from the simulation is the reliable one.

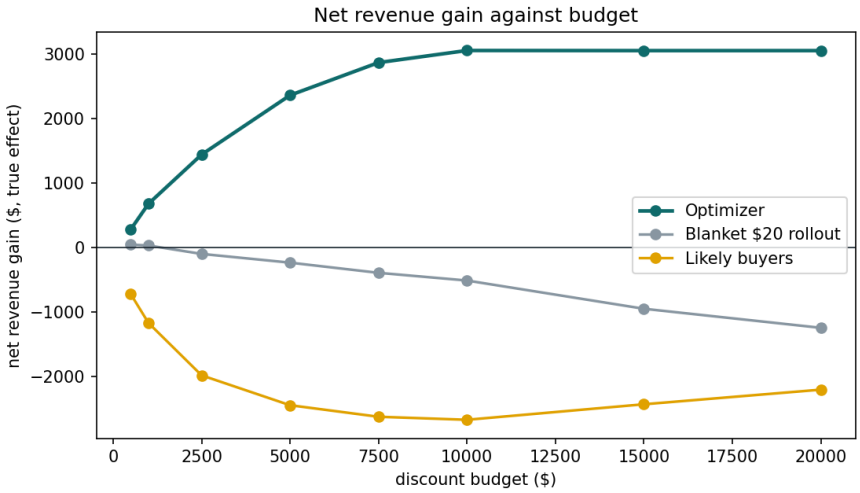


Figure 6. True net revenue against budget. The optimizer levels off near a $10,000 budget once every worthwhile customer has a discount, while the simple strategies keep losing money.

Part 4: Week 4

REST API, drift detection and dashboard polish

Deliverables

- FastAPI service wrapping the causal engine (`backend/api/main.py`), with interactive docs at /docs
- Model packaged as a 3.4 KB JSON file (`backend/api/model_artifact.json`)
- Automated drift detector (`backend/monitoring/drift.py`) with a demo script
- Dashboard Monitoring tab, live budget recalculation through the API, and plain-language summaries
- Dockerfile, API guide, and 45 automated tests

REST API

Endpoint	Purpose
GET /health	Service status and model version
GET /model	Features, test-set accuracy, drift thresholds
POST /predict	Effect and no-discount purchase chance per customer
POST /allocate	Optimal discounts within a budget, comparison strategies and a summary sentence
POST /drift	Drift check of a new batch against the training data

The model is stored as plain numbers: the LinearDML coefficients, a baseline purchase model and a summary of the training data. LinearDML was chosen for the API because it ranked customers best in Week 2 (correlation 0.86 with the true effect) and needs no heavy pickled files. The causal forest is still used for offline evaluation.

Drift detector

The Population Stability Index (PSI) compares each customer feature in a new batch with the training data. Below 0.10 is stable, 0.10 to 0.25 is a warning, and above 0.25 is an alert.

Batch	Result	Detail
No drift (held-out customers)	OK	largest PSI 0.013
Mild drift (35% higher spend)	WARNING	past spend PSI 0.134
Strong drift (50% higher spend, 2 more visits, region mix shifted)	ALERT	visits 0.719, region 0.623, past spend 0.252

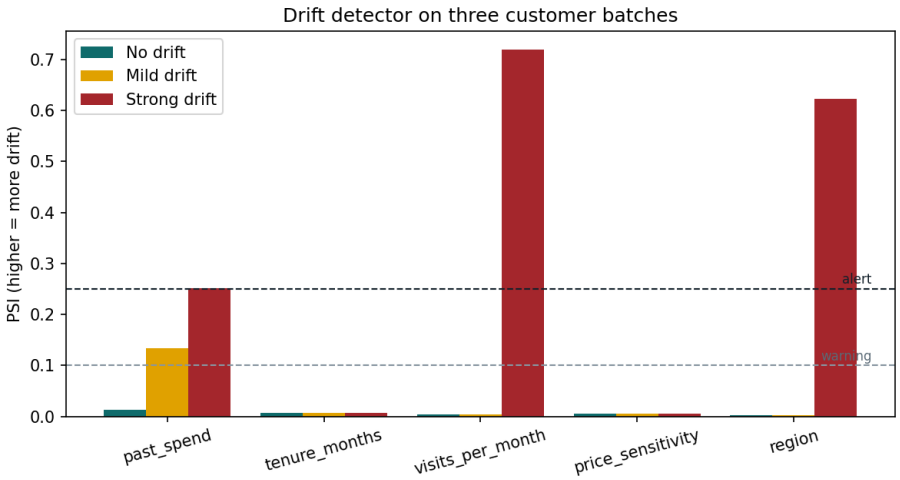


Figure 7. Drift scores for three batches. The detector stays quiet on normal data and escalates as the shift grows.

Dashboard polish

- Allocation tab opens with a plain-English summary and recalculates for any budget when the API is running.
- Monitoring tab shows a traffic-light banner, PSI chart and per-feature table, and can check an uploaded CSV in the browser. The browser calculation matches the Python version exactly on the sample data.

Limitations

- Drift detection watches customer features, not whether the discount effect itself changed. That would need fresh experiment data.
- The Dockerfile has not been built in the project environment, and the dashboard was verified with automated component tests and a build rather than by hand in a browser.
- The API uses a linear effect model, so allocations reflect that assumption.

Overall conclusion

Question	Answer
Does the causal approach recover the true effect?	Yes: +0.146 (95% CI +0.112 to +0.180) against a true +0.127
Does it target customers better than ordinary ML?	Yes: about 97 extra purchases against 57 at random and 44 for a likely-buyer model
Is the result robust?	Refutation tests pass: placebo gives about 0, other changes leave the estimate unchanged
Does it make money under a budget?	Yes: +$2,365 true net revenue, while flat and likely-buyer strategies lose money
Can it be run as a service and monitored?	Yes: REST API plus a drift detector that warns and alerts as behaviour shifts